Lecture 6: Continuous Random Variables, PDFs, CDFs

Jamie Haddock

Table of contents

0.1 Why do we need a new kind of random variable? . . . 1
0.2 Example: the continuous uniform random variable . . . 2
0.3 Example: a PDF can take arbitrarily large values . . . 3
0.4 Expectation of a continuous random variable . . . 3
0.5 The Exponential random variable . . . 4
0.6 Cumulative Distribution Functions . . . 6
0.7 Properties of the CDF . . . 6
0.8 Geometric and Exponential CDFs . . . 7
0.9 Functions of a Continuous RV . . . 9
0.9.1 Derivation of PDF of a function of a cont RV . . . 9

0.1 Why do we need a new kind of random variable?

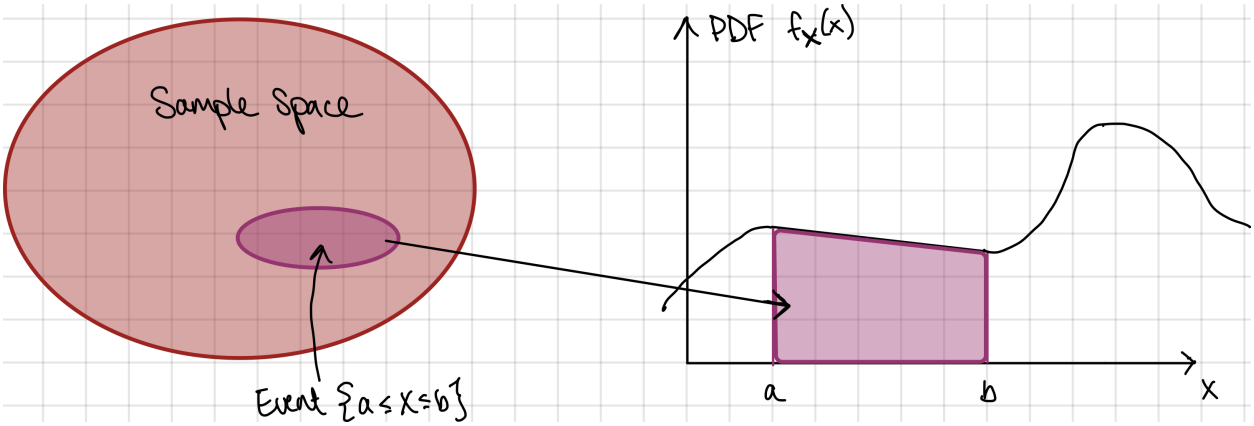
Every random variable so far has been **discrete**: a finite or countably infinite list of possible values, each with its own probability mass. But plenty of quantities we'd like to model — a waiting time, a position, a measurement error — can take *any* real value in an interval. There's no way to list them out one at a time.

Definition: Continuous random variable, PDF

A random variable X is **continuous** if there is a function $f_X(x) \geq 0$, called its **probability density function (PDF)**, such that for every interval $[a, b]$,

$$P(a \leq X \leq b) = \int_a^b f_X(x)\,dx, \qquad \int_{-\infty}^{\infty} f_X(x)\,dx = 1.$$

Instead of bars of probability sitting on individual points (a PMF), probability is now **area under a curve**.



0.2 Example: the continuous uniform random variable

Definition: Uniform(a, b)

$$f_X(x) = \begin{cases} \dfrac{1}{b-a} & a \le x \le b \\ 0 & \text{otherwise.} \end{cases}$$

Every value in $[a, b]$ is "equally likely," in the sense that the density is flat — probability of a sub-interval depends only on its *length*, not its location.

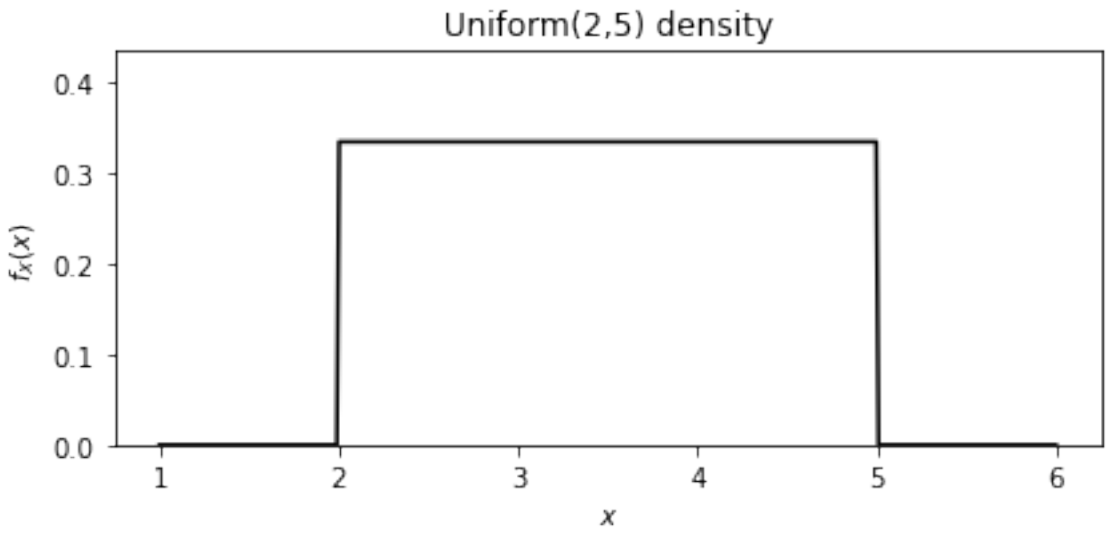
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad

a_u, b_u = 2, 5

def f_uniform(x):
    x = np.asarray(x, dtype=float)
    return np.where((x >= a_u) & (x <= b_u), 1 / (b_u - a_u), 0.0)

xs = np.linspace(a_u - 1, b_u + 1, 400)
fig, ax = plt.subplots(figsize=(6, 3))
ax.plot(xs, f_uniform(xs), color='black')
ax.set_ylim(0, 1 / (b_u - a_u) * 1.3)
ax.set_xlabel('$x$')
ax.set_ylabel('$f_X(x)$')
ax.set_title(f'Uniform({a_u},{b_u}) density')
plt.tight_layout()
plt.show()

total, _ = quad(f_uniform, a_u - 1, b_u + 1)
p_3_4, _ = quad(f_uniform, 3, 4)
print(f"total area:        {total:.4f}")
print(f"P(3 <= X <= 4) =   {p_3_4:.4f}   (should be 1/3 = {1/3:.4f}, since [3,4] is 1/3 of [2,5])")
```




```
total area:         1.0000
P(3 <= X <= 4) =    0.3333   (should be 1/3 = 0.3333, since [3,4] is 1/3 of [2,5])
```

0.3 Example: a PDF can take arbitrarily large values

⚠ A density is *not* a probability

Nothing in the definition requires $f_X(x) \leq 1$ — only that the *total area* equals 1. Squeeze the same unit of probability into a short enough interval and the height of the density can be as large as you like.

ℹ Fact: $P(X = x_0) = 0$ for continuous X

For any single point x_0,

$$P(X = x_0) = \int_{x_0}^{x_0} f_X(x)\,dx = 0,$$

since we're integrating over an interval of zero width. Continuous random variables put *no* probability mass on individual points — only on intervals of positive length.

0.4 Expectation of a continuous random variable

ℹ Definition: Expectation

$$E[X] = \int_{-\infty}^{\infty} x\, f_X(x)\,dx.$$

The rules we've seen previously generalize once we replace summations with integrals!

Theorem: LOTUS for continuous random variables

For any function g,

$$E[g(X)] = \int_{-\infty}^{\infty} g(x)\, f_X(x)\, dx.$$

Definitions: moments and variance

The k-th **moment** of X is $E[X^k]$. **Variance** is defined exactly as in the discrete case,

$$\mathrm{Var}(X) = E\big[(X - E[X])^2\big] = E[X^2] - (E[X])^2,$$

and the shortcut formula's proof used only linearity — nothing about summing versus integrating — so it carries over unchanged.

In-class exercise

Prove that for constants a, b and a continuous random variable X:

$$E[aX + b] = aE[X] + b \qquad \text{and} \qquad \mathrm{Var}(aX + b) = a^2\,\mathrm{Var}(X).$$

Solution: identical to prevous discrete proof, with sums replaced by integrals.

$$E[aX + b] = \int (ax + b)\, f_X(x)\, dx = a \int x\, f_X(x)\, dx + b \int f_X(x)\, dx = aE[X] + b.$$

$$\mathrm{Var}(aX + b) = E\big[(aX + b - aE[X] - b)^2\big] = E\big[a^2(X - E[X])^2\big] = a^2\,\mathrm{Var}(X).$$

Every fact about expectation and variance built on linearity carries over to continuous RVs with zero extra work.

0.5 The Exponential random variable

The continuous "cousin" of the Geometric — instead of counting *trials* until an event, it measures the *waiting time* until an event that can happen at any instant.

Definition: Exponential(λ)

$$f_X(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0, \end{cases} \qquad \lambda > 0.$$

$$E[X] = \frac{1}{\lambda}, \qquad \mathrm{Var}(X) = \frac{1}{\lambda^2}$$

(both derivable by integration by parts) — the same shape of answer as the Geometric(p) mean $1/p$, with λ playing the role of a *continuous rate* in place of p.

Example: customers arrive at a coffee shop at an average rate of $\lambda = 3$ per hour, memorylessly (the time since the last arrival tells you nothing about when the next one comes). The waiting time X until the next customer is Exponential(3), measured in hours, so $E[X] = \frac{1}{3}$ hour $= 20$ minutes.

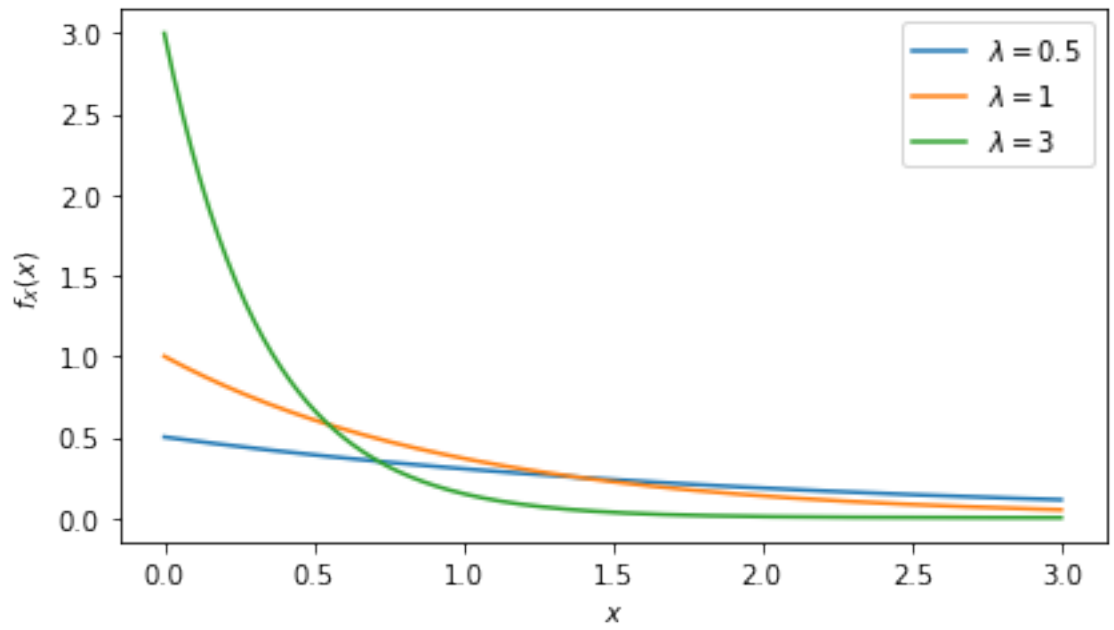
```
def f_exp(x, lam):
    x = np.asarray(x, dtype=float)
    return np.where(x >= 0, lam * np.exp(-lam * x), 0.0)

xs = np.linspace(0, 3, 400)
fig, ax = plt.subplots(figsize=(6, 3.5))
for lam in [0.5, 1, 3]:
    ax.plot(xs, f_exp(xs, lam), label=f'$\\lambda={lam}$')
ax.set_xlabel('$x$')
ax.set_ylabel('$f_X(x)$')
ax.legend()
plt.tight_layout()
plt.show()

lam = 3.0  # the coffee shop: 3 customers/hour
E_exp_num, _ = quad(lambda x: x * f_exp(x, lam), 0, 50)
E_exp2_num, _ = quad(lambda x: x ** 2 * f_exp(x, lam), 0, 50)
Var_exp_num = E_exp2_num - E_exp_num ** 2

rng = np.random.default_rng(151)
wait_times = rng.exponential(1 / lam, size=1_000_000)  # numpy's scale parameter is 1/lambda

print(f"lambda = {lam}")
print(f"E[X]:   formula 1/lambda    = {1/lam:.4f}   quad = {E_exp_num:.4f}   simulated = {wait_times.mean
print(f"Var(X): formula 1/lambda^2 = {1/lam**2:.4f}   quad = {Var_exp_num:.4f}   simulated = {wait_times
```




```
lambda = 3.0
E[X]:   formula 1/lambda    = 0.3333   quad = 0.3333   simulated = 0.3327
Var(X): formula 1/lambda^2 = 0.1111   quad = 0.1111   simulated = 0.1112
```

0.6 Cumulative Distribution Functions

Definition: CDF

For *any* random variable X — discrete or continuous —

$$F_X(x) = P(X \le x).$$

Unlike the PMF or PDF, the CDF is defined the *same way* regardless of type. And each of PMF, PDF, or CDF — whichever applies to X — **unambiguously pins down the entire distribution**: from any one of them, you can recover the others.

0.7 Properties of the CDF

Every CDF, discrete or continuous, satisfies:

- **monotonically nondecreasing:** $x_1 \le x_2 \implies F_X(x_1) \le F_X(x_2)$,
- $F_X(x) \to 0$ as $x \to -\infty$, and $F_X(x) \to 1$ as $x \to \infty$,
- for **discrete** X: F_X is a **piecewise-constant step function** — flat except for a jump of size $p_X(x)$ at every x in the range of X,
- for **continuous** X: F_X is **continuous** everywhere (no jumps at all — consistent with $P(X = x_0) = 0$ for every x_0).

Relation between PDF and CDF, for X continuous:

$$F_X(x) = \int_{-\infty}^{x} f_X(t)\,dt, \qquad \text{equivalently} \qquad f_X(x) = F_X'(x)$$

wherever F_X is differentiable — the Fundamental Theorem of Calculus!

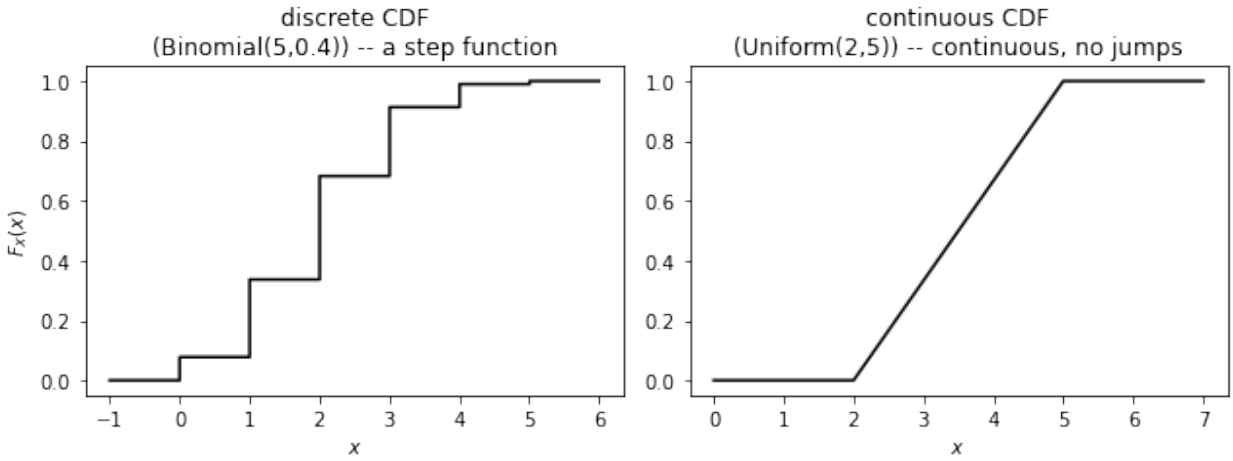
```
from scipy.stats import binom

fig, axes = plt.subplots(1, 2, figsize=(9, 3.5))

# discrete: Binomial(5, 0.4) CDF -- a step function
n_b, p_b = 5, 0.4
xs_d = np.linspace(-1, 6, 700)
cdf_d = binom.cdf(np.floor(xs_d), n_b, p_b)
axes[0].step(xs_d, cdf_d, where='post', color='black')
axes[0].set_title('discrete CDF\n(Binomial(5,0.4)) -- a step function')
axes[0].set_xlabel('$x$')
axes[0].set_ylabel('$F_X(x)$')

# continuous: Uniform(2,5) CDF -- continuous, and (in this special case) piecewise linear
xs_c = np.linspace(0, 7, 400)
cdf_c = np.clip((xs_c - a_u) / (b_u - a_u), 0, 1)
axes[1].plot(xs_c, cdf_c, color='black')
axes[1].set_title('continuous CDF\n(Uniform(2,5)) -- continuous, no jumps')
axes[1].set_xlabel('$x$')

plt.tight_layout()
plt.show()
```



0.8 Geometric and Exponential CDFs

$$X \sim \text{Geometric}(p): \quad F_X(k) = 1 - (1-p)^k, \qquad k = 1, 2, \ldots$$

$$X \sim \text{Exponential}(\lambda): \quad F_X(x) = 1 - e^{-\lambda x}, \qquad x \geq 0.$$

Same shape of formula — because the Exponential really is the **continuous limit** of the Geometric. Chop a unit of time into n tiny slices, let each slice independently "succeed" with probability $p = \lambda/n$, and count *time* (not *trials*) until the first success. As $n \to \infty$, the Geometric CDF — rescaled so trial number k becomes time k/n — converges to the Exponential(λ) CDF.

> Aside
>
> Both distributions are **memoryless** — the property we used to derive the Geometric's mean carries over exactly to the Exponential, and we'll put it to work again later this course.

```
lam = 2.0
t = np.linspace(0, 3, 400)
exp_cdf = 1 - np.exp(-lam * t)

fig, ax = plt.subplots(figsize=(6, 3.5))
ax.plot(t, exp_cdf, 'k--', lw=2, label=f'Exponential($\\lambda={lam}$) CDF')

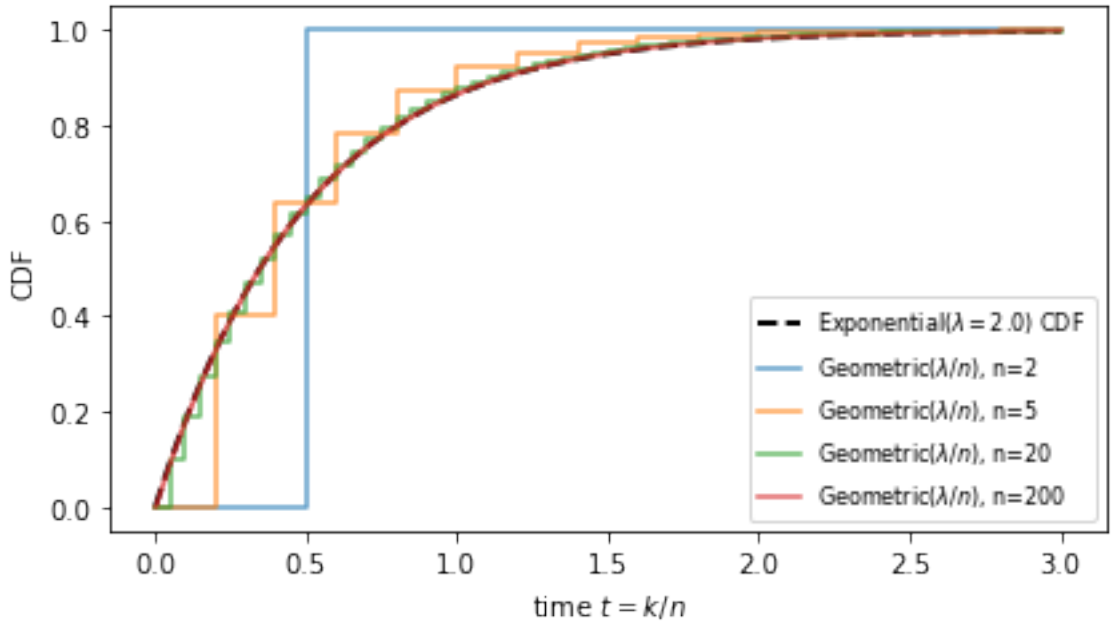
for n in [2, 5, 20, 200]:
    p = lam / n
    k_vals = np.arange(0, int(3 * n) + 1)
    geo_cdf = 1 - (1 - p) ** k_vals
    ax.step(k_vals / n, geo_cdf, where='post', alpha=0.7, label=f'Geometric($\\lambda/n$), n={n}')

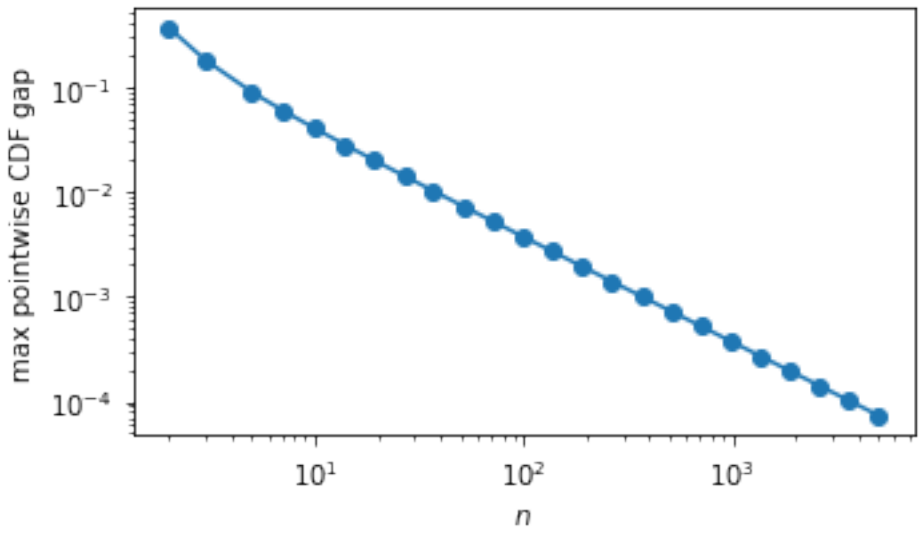
ax.set_xlabel('time $t = k/n$')
ax.set_ylabel('CDF')
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()

# max pointwise CDF gap, shrinking as n grows
n_range = np.unique(np.geomspace(2, 5000, 25).astype(int))
```

```
max_gaps = []
for n in n_range:
    p = lam / n
    k_vals = np.arange(0, int(5 * n) + 1)
    geo_cdf = 1 - (1 - p) ** k_vals
    t_vals = k_vals / n
    exp_at_t = 1 - np.exp(-lam * t_vals)
    max_gaps.append(np.max(np.abs(geo_cdf - exp_at_t)))

fig2, ax2 = plt.subplots(figsize=(5, 3))
ax2.loglog(n_range, max_gaps, 'o-')
ax2.set_xlabel('$n$')
ax2.set_ylabel('max pointwise CDF gap')
plt.tight_layout()
plt.show()
```





0.9 Functions of a Continuous RV

While we have LOTUS for a continuous RV, we will occasionally want the PDF of a function of a continous random variable.

Theorem: PDF of a function of a continuous RV

Let X be a continuous random variable and let $Y = g(X)$, where g is invertible and differentiable, with inverse $h = g^{-1}$. Then Y is continuous with density

$$f_Y(y) = f_X(h(y))\,|h'(y)|.$$

0.9.1 Derivation of PDF of a function of a cont RV

Setup

Let $Y = g(X)$ where g is invertible and differentiable, with inverse $h = g^{-1}$. Since g is invertible and continuous, it is either strictly increasing everywhere or strictly decreasing everywhere.

Case 1: g increasing (so h is increasing too, $h'(y) > 0$).

$$F_Y(y) = P(Y \le y) = P(g(X) \le y) = P(X \le h(y)) = F_X(h(y)).$$

Differentiating both sides in y (chain rule):

$$f_Y(y) = f_X(h(y))\,h'(y) = f_X(h(y))\,|h'(y)|.$$

Case 2: g decreasing (so h is decreasing too, $h'(y) < 0$).

$$F_Y(y) = P(g(X) \le y) = P(X \ge h(y)) = 1 - F_X(h(y)).$$

Differentiating:

$$f_Y(y) = -f_X(h(y))\,h'(y) = f_X(h(y))\,|h'(y)|.$$

Intuition: matching probability mass

An interval $[x, x+dx]$ maps under g to an interval of length $|h'(y)|\,dy$ near $y = g(x)$, and g carries *all* the probability mass from $[x, x+dx]$ into that image interval. It is the same probability mass, but the interval has a different width, so the density has to rescale by the local stretch factor $|h'(y)|$:

$$f_Y(y)\,|dy| = f_X(x)\,|dx| \quad\Longrightarrow\quad f_Y(y) = f_X(h(y))\,|h'(y)|.$$

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch

# ---- house-style palette (teal accent, matches the callout-tip green) ----
ACCENT = "#2f9e6f"       # shaded "matched mass" regions
ACCENT_DARK = "#1b6e4f"
CURVE = "#1f3a5f"        # density curves
GRID = "#c8ccd1"
ARROW = "#555555"

# ---- the running example ----
def g(x):
    return x ** 2

def h(y):
    return np.sqrt(y)

def hprime(y):
    return 1.0 / (2 * np.sqrt(y))

def f_X(x):
    return np.ones_like(x)

def f_Y(y):
    return hprime(y)  # since f_X(h(y)) = 1

# the highlighted interval in X-space
x0, dx = 0.6, 0.12
x1 = x0 + dx
y0, y1 = g(x0), g(x1)   # exact image interval; dy = y1 - y0 (not simply g'(x0)*dx)
dy = y1 - y0

area_left = dx * 1.0                        # rectangle: width * height
area_right = np.sqrt(y1) - np.sqrt(y0)      # exact integral of 1/(2 sqrt y)
assert abs(area_left - area_right) < 1e-12

fig, axes = plt.subplots(1, 2, figsize=(12, 4.9))
fig.subplots_adjust(wspace=0.55, top=0.80, bottom=0.30)

# ---------------- LEFT PANEL: X-space ----------------
axL = axes[0]
```

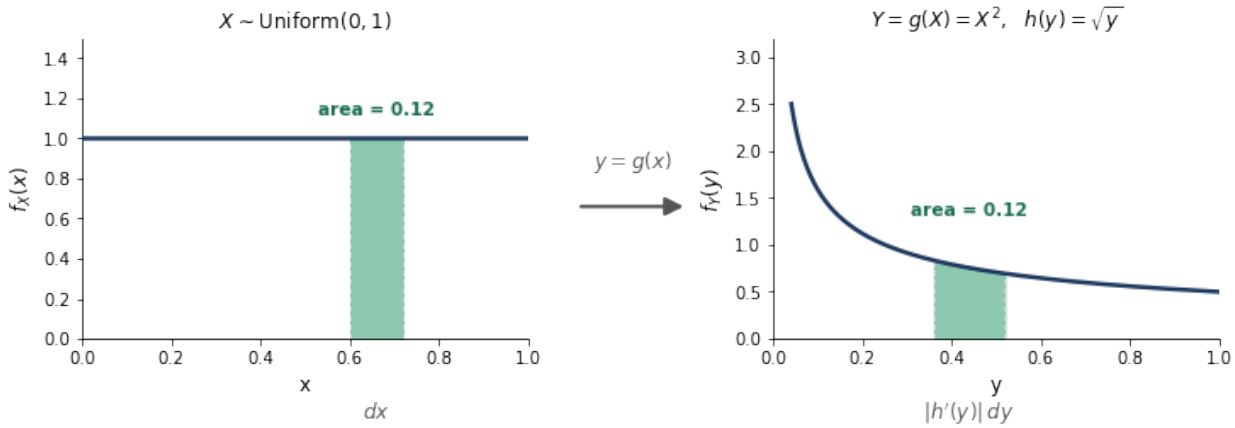
```
xs = np.linspace(0, 1, 400)
axL.plot(xs, f_X(xs), color=CURVE, lw=2.5, zorder=3)
axL.fill_between([x0, x1], [0, 0], [1, 1], color=ACCENT, alpha=0.55, zorder=2,
                 label=f"area = {area_left:.2f}")
axL.set_ylim(0, 1.5)
axL.set_xlim(0, 1)
axL.set_xlabel("x", fontsize=12)
axL.set_ylabel(r"$f_X(x)$", fontsize=12)
axL.set_title(r"$X \sim \mathrm{Uniform}(0,1)$", fontsize=12)
axL.axvline(x0, color=GRID, lw=1, ls="--", ymax=1/1.5)
axL.axvline(x1, color=GRID, lw=1, ls="--", ymax=1/1.5)

axL.text((x0 + x1) / 2, -0.20, r"$dx$", ha="center", va="top",
         transform=axL.get_xaxis_transform(), fontsize=12, color=ARROW)
axL.text((x0 + x1) / 2, 1.12, f"area = {area_left:.2f}", ha="center", fontsize=10.5,
         color=ACCENT_DARK, fontweight="bold")
for spine in ["top", "right"]:
    axL.spines[spine].set_visible(False)

# ---------------- RIGHT PANEL: Y-space ----------------
axR = axes[1]
ys = np.linspace(0.04, 1, 400)
axR.plot(ys, f_Y(ys), color=CURVE, lw=2.5, zorder=3)
yy = np.linspace(y0, y1, 100)
axR.fill_between(yy, 0, f_Y(yy), color=ACCENT, alpha=0.55, zorder=2)
axR.set_ylim(0, 3.2)
axR.set_xlim(0, 1)
axR.set_xlabel("y", fontsize=12)
axR.set_ylabel(r"$f_Y(y)$", fontsize=12)
axR.set_title(r"$Y = g(X) = X^2,\ \ h(y)=\sqrt{y}$", fontsize=12)
ymax_frac = f_Y(np.array([y1]))[0] / 3.2
axR.axvline(y0, color=GRID, lw=1, ls="--", ymax=f_Y(np.array([y0]))[0] / 3.2)
axR.axvline(y1, color=GRID, lw=1, ls="--", ymax=ymax_frac)

axR.text((y0 + y1) / 2, -0.20, r"$|h'(y)|\,dy$", ha="center", va="top",
         transform=axR.get_xaxis_transform(), fontsize=12, color=ARROW)
axR.text((y0 + y1) / 2, f_Y(np.array([(y0+y1)/2]))[0] + 0.55, f"area = {area_right:.2f}",
         ha="center", fontsize=10.5, color=ACCENT_DARK, fontweight="bold")
for spine in ["top", "right"]:
    axR.spines[spine].set_visible(False)

# ---------------- connecting arrow across the two panels ----------------
arrow = FancyArrowPatch((0.465, 0.52), (0.535, 0.52), transform=fig.transFigure,
                        mutation_scale=22, color=ARROW, lw=2.2, shrinkA=0, shrinkB=0,
                        arrowstyle="-|>")
fig.patches.append(arrow)
fig.text(0.5, 0.585, r"$y = g(x)$", ha="center", fontsize=12, color=ARROW);
```


$X \sim \text{Uniform}(0, 1)$
area = 0.12
$f_X(x)$
x
dx
$y = g(x)$
$Y = g(X) = X^2, \quad h(y) = \sqrt{y}$
area = 0.12
$f_Y(y)$
y
$|h'(y)|\, dy$